

NearMiss

- In this method, the majority class instances that are near the minority class instances are removed.
- There are three versions of this algorithm, and each version has a different criterion for selecting which majority class instances to keep and which ones to discard.
 - **NearMiss-1:** The data is balanced by selecting the samples from the majority class for which the average distance of the k nearest samples of the minority class is the smallest.
 - **NearMiss-2:** The data is balanced by selecting the samples from the majority class for which the average distance to the farthest samples of the minority class is the smallest.
 - **NearMiss-3:** This is a 2-step algorithm: first, the m nearest-neighbours are kept for each minority sample; then, the majority samples for which the average distance to the k nearest neighbours is the largest are selected.
- The logic is:
 - This algorithm has two important parameters — version and k neighbours.
 - It calculates the distance between all the points in the larger class with the points in the smaller class.
 - Elimination is done based on the passed parameter.
 - **NearMiss-1:** This version keeps instances from the majority class that are closest to the instances in the minority class.
 - **NearMiss-2:** This version removes instances from the majority class that are farthest from the minority class.
 - **NearMiss-3:** This version is somewhat opposite to NearMiss-1; it keeps instances from the majority class that are farthest from the instances in the minority class.
- The sample code is:

```
from collections import Counter
from sklearn.datasets import make_classification
from sklearn.utils import resample
from imblearn.under_sampling import NearMiss
from matplotlib import pyplot
from numpy import where

X, y = make_classification(n_classes=2, class_sep=2,
weights=[0.1, 0.9], n_features=4, n_clusters_per_class=1,
n_samples=1000, random_state=10)

# summarise class distribution
print('Original dataset shape %s' % Counter(y))
Original dataset shape Counter({1: 894, 0: 106})
```

We will now perform downsampling:

```
cc = NearMiss()
X_Res, y_res = cc.fit_resample(X, y)
print('Resampled dataset shape %s' % Counter(y_res))
Resampled dataset shape Counter({0: 106, 1: 106})

# scatter plot of examples by class label
for label, _ in Counter(y_res).items():
    row_ix = where(y_res == label)[0]
    pyplot.scatter(X_Res[row_ix, 0], X_Res[row_ix, 1],
label=str(label))
pyplot.legend()
pyplot.show()
```

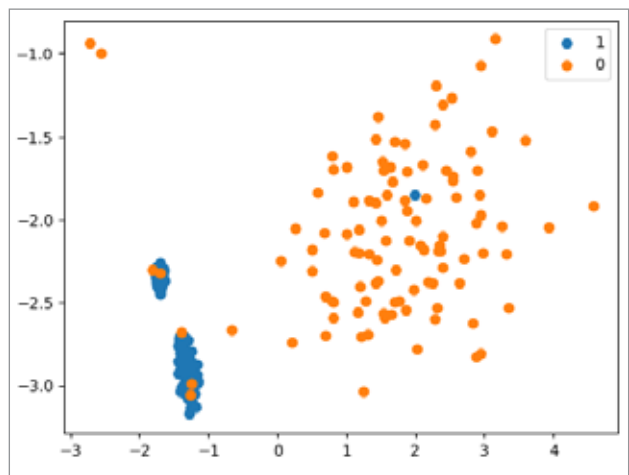


Figure 5: Scatter plot of examples by class label downsampled post using NearMiss

Tomek links (T-links)

- Here a pair of data points from different classes (nearest-neighbours) are dropped — the objective is to drop the sample that corresponds to the majority and thereby minimise the count of the dominating label.
- This also increases the border space between the two labels and thus enhances the accuracy of the performance.
- Here's the logic.
 - Identifying Tomek links:
 - A Tomek link is a pair of instances (x, y) where x is from the majority class and y is from the minority class, leaving no other smaller instance between.
 - Removing instances in Tomek links:
 - Once Tomek links are identified, the instances from the majority class in these links are removed.
 - This helps in creating a cleaner and more separable boundary between the classes.
 - Resulting effect: